Kegiatan Belajar 4F.4: Penguncian Berkas dan Persiapan Transaksi

Hari 15 · 8 JP · UK 1.1, 1.3, 1.4, dan 1.5 · Asesmen Modul 4F

Tujuan KB 4F.4. Setelah mengikuti KB ini, Anda mampu menjelaskan dan menerapkan penguncian berkas yang lebih ketat, merancang struktur **`transaksi.json`** dan **`pengguna.json`**, membuat fungsi pembuat kode transaksi acak, dan menjalani asesmen Modul 4F.

Kegiatan	**JP**
Mengapa `LOCK_EX` pada penulisan saja belum selalu cukup	1
Praktik: fungsi baca-tulis dengan kunci penuh	2
Rancangan `transaksi.json` dan `pengguna.json`	1
Kode transaksi acak; README; organisasi folder	2
Asesmen Modul 4F	2
Jumlah	**8**

Materi 1: Keterbatasan Penguncian Sederhana

`tulisJson()` di **KB 4F.2** memakai **`LOCK_EX`** **hanya** selama proses penulisan. Tapi bayangkan urutan kejadian berikut, dengan dua proses (**P1** dan **P2**) yang **sama-sama** ingin mengurangi stok produk yang sama:

```
Waktu   P1 (checkout pelanggan A)         P2 (checkout pelanggan B)
-----   --------------------------        --------------------------
t1      baca produk.json (stok: 5)
t2                                        baca produk.json (stok: 5)
t3      stok baru = 5 - 2 = 3
t4                                        stok baru = 5 - 1 = 4
t5      tulis produk.json (stok: 3)
t6                                        tulis produk.json (stok: 4)   <- MENIMPA hasil P1!
```

Hasil akhirnya stok tertulis **4**, padahal seharusnya **2** (5 dikurangi 2 dikurangi 1). **`LOCK_EX`** pada `tulisJson()` **tidak mencegah ini**, karena P1 dan P2 sama-sama sempat **membaca** sebelum salah satu dari mereka sempat menulis — kuncinya datang terlambat. Inilah yang disebut ***race condition*** (kondisi balapan), dan ini **persis** masalah yang di Bagian B (basis data) diselesaikan otomatis oleh `beginTransaction`/`commit`/`rollBack`.

Solusi untuk Bagian B2: kunci berkas **sebelum** membaca, tahan kuncinya sepanjang proses baca-ubah-tulis, baru lepaskan setelah selesai menulis. Proses lain yang mencoba mengunci berkas yang sama akan **menunggu** sampai kunci dilepas.

Praktik 1: Fungsi Baca-Tulis dengan Kunci Penuh

Tambahkan di `api/lib/json_store.php`:

```php
<?php
// Versi "ketat": mengunci berkas SEPANJANG proses baca-ubah-tulis,
// dipakai khusus untuk operasi yang benar-benar tidak boleh tertimpa
// proses lain (mis. checkout yang mengurangi stok -- lihat Modul 5F).

function bukaJsonTerkunci(string $path, &$handle): array {
    $handle = fopen($path, "c+");   // "c+": buat jika belum ada, boleh baca & tulis
    flock($handle, LOCK_EX);        // kunci DULU, SEBELUM membaca apa pun
    $ukuran = filesize($path) ?: 0;
    $teks = $ukuran > 0 ? fread($handle, $ukuran) : "";
    $data = $teks !== "" ? json_decode($teks, true) : [];
    return $data ?? [];
}

function tutupJsonTerkunci($handle, array $data): void {
    $teks = json_encode($data, JSON_PRETTY_PRINT | JSON_UNESCAPED_SLASHES);
    rewind($handle);
    ftruncate($handle, 0);          // kosongkan isi lama sebelum menulis yang baru
    fwrite($handle, $teks);
    fflush($handle);
    flock($handle, LOCK_UN);        // lepas kunci SETELAH selesai menulis
    fclose($handle);
}
```

Cara memakainya (contoh konsep, dipraktikkan penuh di Modul 5F):

```php
<?php
$data = bukaJsonTerkunci(__DIR__ . "/../data/produk.json", $handle);
// ... ubah $data di sini (mis. kurangi stok salah satu produk) ...
tutupJsonTerkunci($handle, $data);
```

Selama baris-baris "ubah `$data`" berjalan, **proses lain yang mencoba `bukaJsonTerkunci()` pada berkas yang sama akan menunggu** sampai `tutupJsonTerkunci()` dipanggil — inilah yang menutup celah *race condition* di Materi 1.

::: {custom-style="Catatan Kotak"} **Catatan kejujuran.** Pola ini cukup kuat untuk skala kelas dan proyek kecil-menengah — tapi tetap bukan jaminan seketat transaksi basis data sungguhan (misalnya, tidak menangani kegagalan di tengah proses dengan sebaik `rollBack`). Untuk aplikasi yang benar-benar menangani volume transaksi tinggi dan data krusial, basis data sungguhan (seperti pada Bagian B) tetap pilihan yang lebih tepat. Memahami batasan ini **adalah bagian dari pembelajaran**, bukan kekurangan modul ini. :::

Praktik 2 (Opsional): Melihat Penguncian Bekerja Secara Langsung

Buat `latihan-php/uji_locking.php`:

```php
<?php
// Jalankan skrip ini di DUA jendela terminal secara hampir bersamaan
// (lewat CLI: php uji_locking.php) untuk melihat proses kedua MENUNGGU.

$path = __DIR__ . "/uji.lock";
if (!file_exists($path)) {
    file_put_contents($path, "");
}

$handle = fopen($path, "c+");
```

```
echo "Menunggu kunci...\n";
flock($handle, LOCK_EX);
echo "Kunci didapat. Menahan selama 5 detik...\n";
sleep(5);
flock($handle, LOCK_UN);
fclose($handle);
echo "Kunci dilepas.\n";
```

Jalankan `php latihan-php/uji_locking.php` di satu jendela terminal, lalu **segera** jalankan perintah yang sama di jendela terminal kedua. Proses kedua akan mencetak "Menunggu kunci..." dan **berhenti di situ** selama proses pertama masih menahan kunci (5 detik) — bukti nyata bahwa `flock()` benar-benar memblokir proses lain, bukan sekadar teori.

Materi 2: Rancangan `transaksi.json` dan `pengguna.json`

Satu pelanggan bisa memesan **banyak** produk sekaligus. Pada basis data, ini butuh dua tabel terpisah (`transaksi` dan `transaksi_detail`). Pada **JSON**, cukup **satu array bersarang** — rincian produk langsung dimasukkan ke dalam setiap objek transaksi:

```
[
  {
    "id": 1,
    "kode_transaksi": "KN-A1B2C3D4",
    "nama_pemesan": "...",
    "kontak": "...",
    "total": 230000,
    "status": "baru",
    "dibuat_pada": "2026-10-10 14:32:00",
    "items": [
      { "produk_id": 1, "jumlah": 2, "harga_saat_beli": 85000 },
      { "produk_id": 5, "jumlah": 1, "harga_saat_beli": 60000 }
    ]
  }
]
```

`harga_saat_beli` tetap disimpan terpisah dari harga produk saat ini (sama seperti Bagian B) — jika harga produk berubah di masa depan, struk pesanan **lama** harus tetap menampilkan harga saat pesanan itu dibuat.

`pengguna.json` (dibangun penuh di Modul 6F):

```
[
  { "id": 1, "nama": "Admin Kopi Nusantara", "username": "admin", "kata_sandi": "(hasil
password_hash)" }
]
```

Praktik 3: Membuat Berkas Data dan Kode Transaksi

Buat `data/transaksi.json` berisi array kosong:

```
[]
```

Buat `data/pengguna.json` berisi array kosong (diisi lewat skrip di Modul 6F):

```
[]
```

Keduanya otomatis ikut terlindungi oleh **`.htaccess`** yang sudah dipasang di **KB 4F.2** (satu `.htaccess` melindungi **seluruh** isi folder `data/`, bukan per berkas).

Tambahkan di `api/lib/json_store.php` (atau berkas fungsi terpisah bila Anda lebih suka):

```
<?php
function buatKodeTransaksi(): string {
    return "KN-" . strtoupper(bin2hex(random_bytes(4)));
}
```

`random_bytes(4)` menghasilkan 4 byte acak kriptografis, `bin2hex()` mengubahnya jadi teks heksadesimal yang aman dipakai di URL — fungsi ini **belum** disambungkan ke proses checkout sungguhan (itu pekerjaan Modul 5F ya guys), hari ini cukup memahami dan mencobanya berdiri sendiri lewat `latihan-php/`.

Materi 3: README dan Organisasi Folder

Perbarui `README.md`, tambahkan bagian baru:

~~~markdown
## Backend (Bagian B2 — Flat-File JSON)

Proyek ini memiliki backend PHP, **tanpa basis data** — seluruh data
tersimpan di berkas JSON pada folder `data/`.

### Struktur Data

- `data/produk.json` — daftar produk (dan info situs/menu)
- `data/transaksi.json` — daftar transaksi (rincian produk bersarang di dalamnya)
- `data/pengguna.json` — daftar akun admin
- `data/.htaccess` — memproteksi folder ini dari akses langsung lewat browser

### Endpoint API

| Endpoint | Metode | Keterangan |
|---|---|---|
| `api/produk.php` | GET | Seluruh data produk, situs, dan menu |

### Menjalankan Proyek

1. Pastikan Laragon aktif (Apache + PHP 8.5). **Tidak perlu MariaDB.**
2. Buka `http://localhost/kopi-nusantara/`.
~~~

Asesmen Modul 4F

Asesmen dilaksanakan di akhir Hari 14 dengan tiga bagian:

1. **Unjuk kerja:** instruktur memeriksa proyek Kopi Nusantara **dan** proyek tema sendiri Anda dengan **Lembar Asesmen (bagian F)**, termasuk demonstrasi singkat: tunjukkan `data/produk.json` tidak bisa diakses langsung, endpoint berjalan normal, dan Kopi Nusantara berjalan dari API.
2. **Soal konsep *race condition*:** diberikan skenario baru (mis. "dua admin mengubah data produk yang sama nyaris bersamaan lewat dashboard"), peserta diminta menjelaskan apa yang bisa salah dan bagaimana `bukaJsonTerkunci()`/`tutupJsonTerkunci()` mencegahnya.

3. **Tes tertulis:** 8 butir dari Lembar Kerja 4F (lihat bagian E).

Rangkuman KB 4F.4

- `LOCK_EX` pada penulisan saja tidak menutup celah *race condition* jika dua proses sama-sama sempat membaca sebelum salah satu menulis.
- Kunci yang dipasang **sebelum** membaca dan dilepas **setelah** menulis (`bukaJsonTerkunci`/`tutupJsonTerkunci`) menutup celah itu, dengan proses lain menunggu sampai kunci dilepas.
- `transaksi.json` memakai array bersarang (`items`) sebagai pengganti tabel rincian terpisah — lebih sederhana dibanding dua tabel relasional.
- Pendekatan ini punya batasan jujur dibanding transaksi basis data sungguhan, dan itu bagian dari pemahaman yang perlu dimiliki, bukan ditutupi.

Cek Kemampuan KB 4F.4

- ☐ Saya dapat menjelaskan *race condition* dengan skenario konkret (dua proses, urutan waktu).
- ☐ `bukaJsonTerkunci()`/`tutupJsonTerkunci()` sudah dicoba dan dipahami cara kerjanya.
- ☐ `data/transaksi.json` dan `data/pengguna.json` sudah dibuat (berisi `[]`).
- ☐ `README.md` saya mencakup bagian backend yang lengkap.

Kesalahan umum: memakai `tulisJson()` versi sederhana (KB 4F.2) untuk operasi yang seharusnya memakai versi terkunci penuh (mis. nanti saat checkout); lupa `ftruncate($handle, 0)` sebelum menulis ulang (sisa teks lama bisa tercampur dengan data baru); lupa `flock($handle, LOCK_UN)` di akhir (kunci tertinggal, proses lain menunggu selamanya).